FEDERATED LEARNING FOR IOT AIR QUALITY MONITORING

Federated PM2.5 Forecasting on IoT Air-Quality Nodes

A CNN-RNN hybrid model trained at the edge and combined with Federated Averaging, so raw sensor data never leaves the device



TEAM

Pranav G
231CS143

Datta L
231CS119

Harsh K
231CS126

Based on: Utku et al., Atmosphere 2025, 16(9), 1003

From one central model to many edge learners

THE PAPER

CNN-RNN hybrid for next-hour PM2.5

Paper dataset	MAE	R2
India	1.620	**0.995**
Milan	2.511	**0.834**
Frankfurt	0.516	**0.995**

Limits the authors state

- Uses PM2.5 history only, no weather inputs
- Tested offline, not on live, noisy sensor streams

OUR EXTENSION

Federated learning on IoT nodes

Federated training
Nodes share model weights, never raw readings

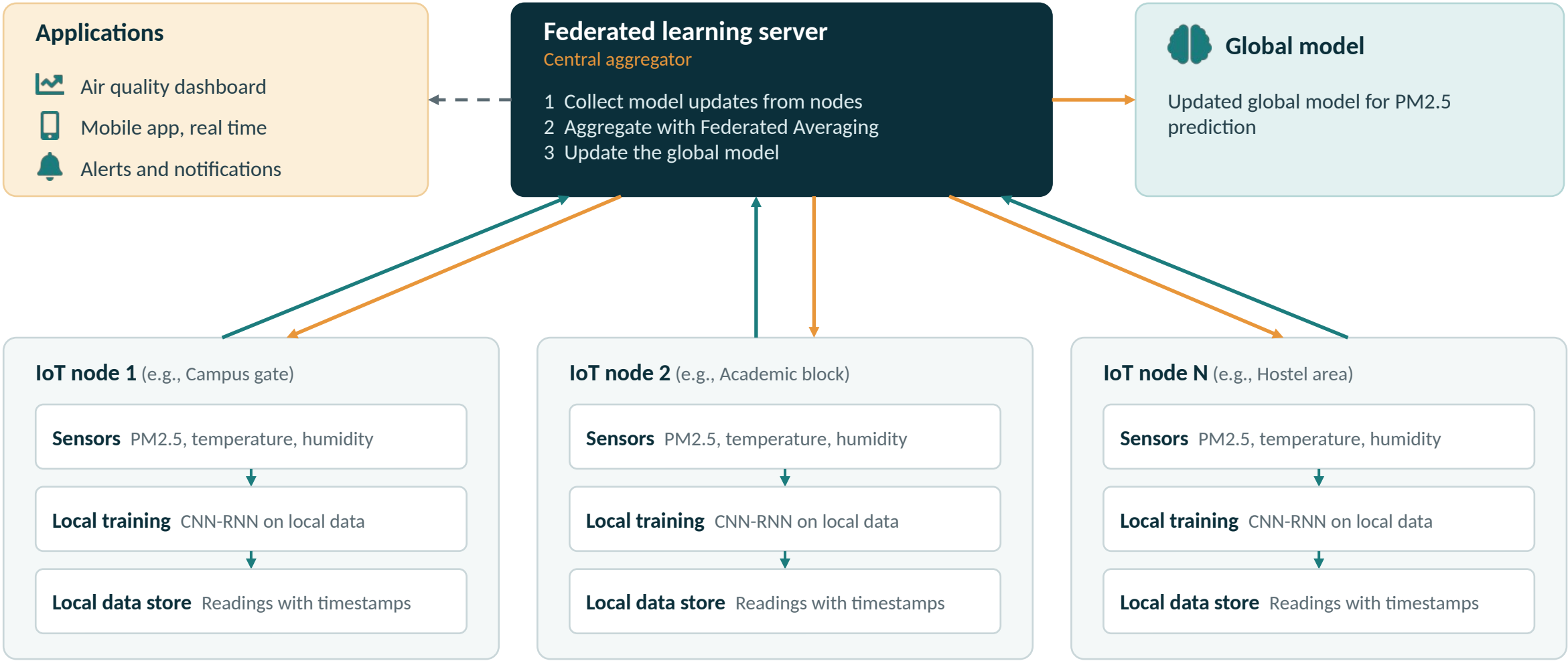
Richer inputs
PM2.5 plus temperature and humidity at every node

Edge deployment
Training and inference on Raspberry Pi-class boards

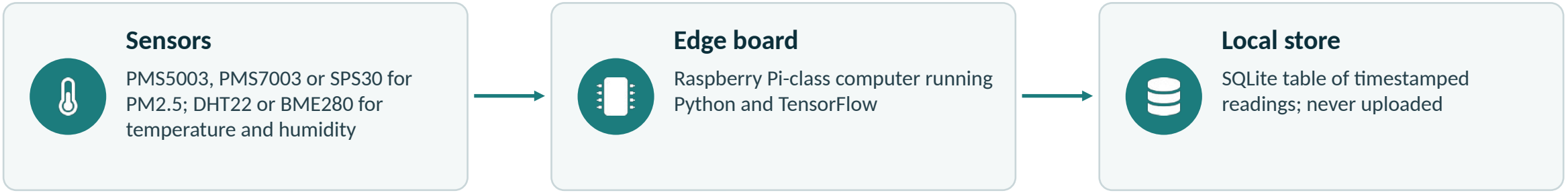
Test bed
Beijing PM2.5 data: 43,824 hourly rows, 2010 to 2014

Paper figures come from its India, Milan and Frankfurt datasets, not from the Beijing dataset used in this project.

Architecture: raw data never leaves the node



Sense: each node collects and prepares its own data



Signal mapping: hardware to dataset

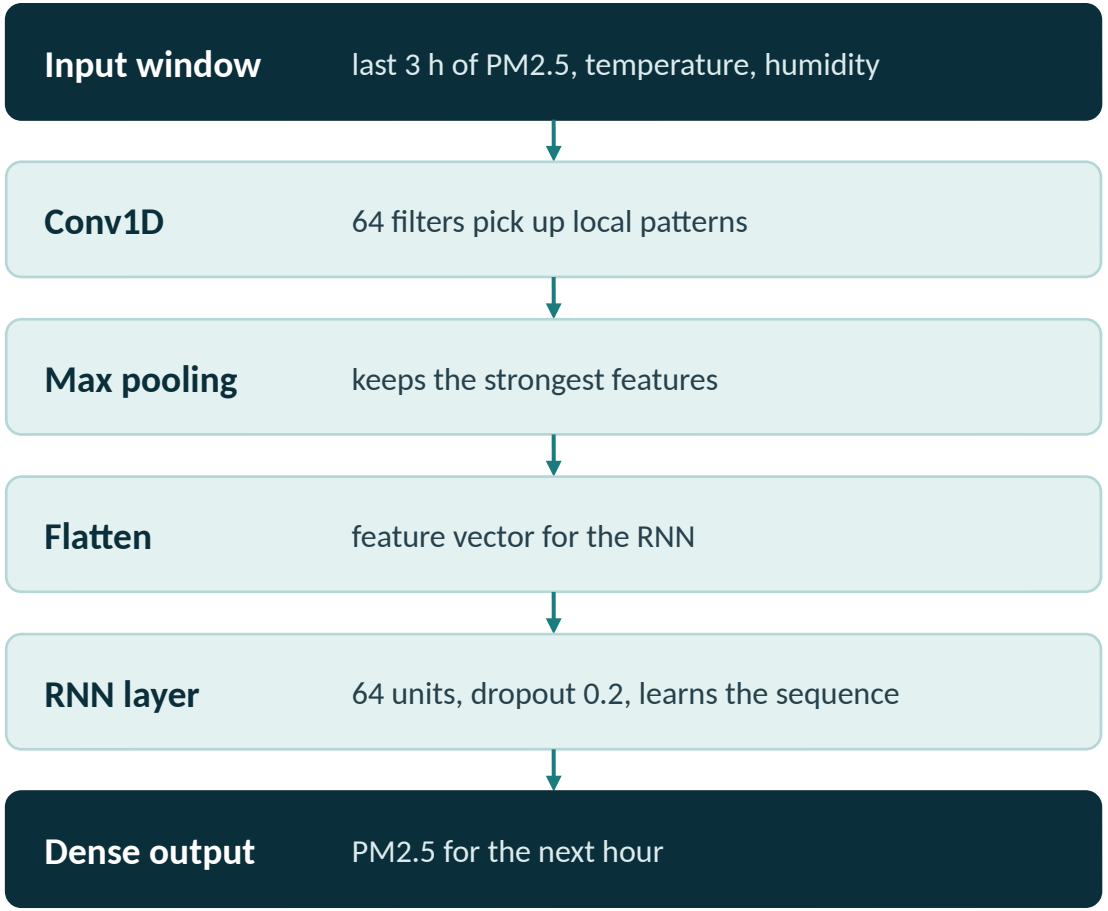
Signal	Hardware (from diagram)	Beijing dataset column
PM2.5	PMS5003, PMS7003, SPS30	pm2.5 (target)
Temperature	DHT22, BME280	TEMP
Humidity	DHT22, BME280	derived from TEMP and DEWP
Time	Board clock (NTP)	year, month, day, hour

The Beijing data has no humidity column, so we derive relative humidity from temperature and dew point (Magnus formula) while nodes are simulated. Real nodes read humidity directly.

Prepare locally on the node

1. Fill missing readings (about 2,000 NA hours in the Beijing data)
2. Remove outliers
3. Min-Max scale using ranges shared by all nodes
4. Slide a 3-hour window to predict the next hour

Train: a lightweight CNN-RNN runs on every node



Hybrid design from Utku et al. (2025)

~38K

trainable parameters in the paper's CNN-RNN

2.4 s

per epoch on the paper's RTX 3060; our Pi timing is still to be measured

What we change from the paper

- Inputs grow from PM2.5 only to PM2.5, temperature and humidity; the paper lists weather variables as future work
- Each node trains for a few local epochs per round instead of one central run
- Start from the paper's settings (Adam, learning rate 0.001, early stopping) and tune on validation data

Aggregate: one federated round, step by step

1

Broadcast

Server sends the current global weights to every available node.

2

Train locally

Each node trains on its own readings for a few epochs.

3

Upload weights

Nodes return updated weights only, about 150 KB each.

4

Average

Server combines them with FedAvg and starts the next round.

repeat until validation error stops improving

FedAvg: $w_{global} = \Sigma_k \, (n_k / n) \cdot w_k$

n_k = samples at node k, n = total samples, w_k = weights returned by node k

Privacy

Raw readings never leave the node. Weights can still leak information, so secure aggregation or differential privacy is a planned extension.

Non-IID data

Nodes see different pollution patterns. If plain FedAvg drifts, we try FedProx.

Offline nodes

A node that is offline skips the round and the others carry on.

Tech stack, layer by layer

SENSE
Edge node

PMS5003 / PMS7003 / SPS30 | DHT22 / BME280 | Raspberry Pi-class board | Python | SQLite

Same sensors as the architecture diagram; the board does the local work.

TRAIN
Local learning

TensorFlow / Keras | NumPy, pandas, scikit-learn | Conv1D + RNN model

Same framework family as the paper; the model is small, so on-device training is feasible to test.

AGGREGATE
Server

Flower (flwr), FedAvg | gRPC + TLS | Docker | Campus server or cloud VM

Flower is framework-agnostic, ships FedAvg and has embedded-device examples.

SERVE
Applications

TensorFlow Lite on nodes | Streamlit dashboard | Mobile web app | Telegram or Firebase alerts

The updated global model returns to the nodes; people see it on the dashboard and phone.

How we will get there

PHASE 1

Simulate

Clean the Beijing data, split it into 5 nodes (one per year, 2010 to 2014) and train the CNN-RNN centrally as the reference.

PHASE 2

Federate

Run Flower with FedAvg on the simulated nodes and compare against nodes training alone.

PHASE 3

Deploy

Move the client onto Raspberry Pi nodes with real sensors; connect the dashboard and alerts.

What we will measure

Centralized: all data pooled

Local only: no sharing

Federated: FedAvg

Compared on RMSE, MAE and R2 (the paper's metrics), plus update size and training time per round on the Pi.

Known limit

The Beijing data comes from a single station, so the nodes in Phases 1 and 2 are simulated splits. Real multi-site behavior is only tested in Phase 3.

Sources: Utku et al., Atmosphere 2025, 16(9), 1003 (doi 10.3390/atmos16091003) | Beijing PM2.5 dataset, Kaggle: ineubytes/pm25-airpolution-dataset